Name: Khadija Arefin Meem

IT21059

Block Cipher Modes of Operation:-

A block cipher is an encryption algorithm that processes data in fixed-size blocks rather than one bit at a time. Block cipher modes of operation define how to securely encrypt and decrypt large amounts of data using a block cipher.

Here are a few common modes:

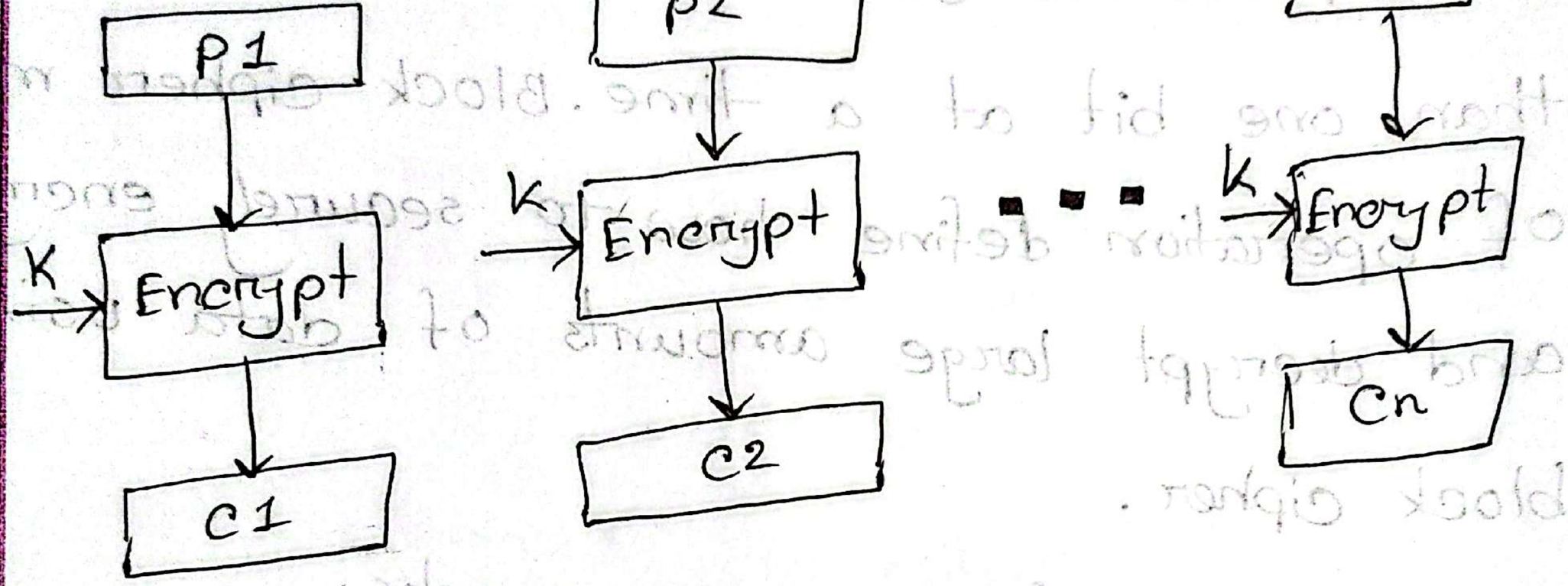
Electronic Code Block (ECB):

The electronic codebook is the easiest block cipher mode of functioning. It is easier because of the direct encryption of each block of input plaintext and output is in the form of blocks of encrypted ciphertext.

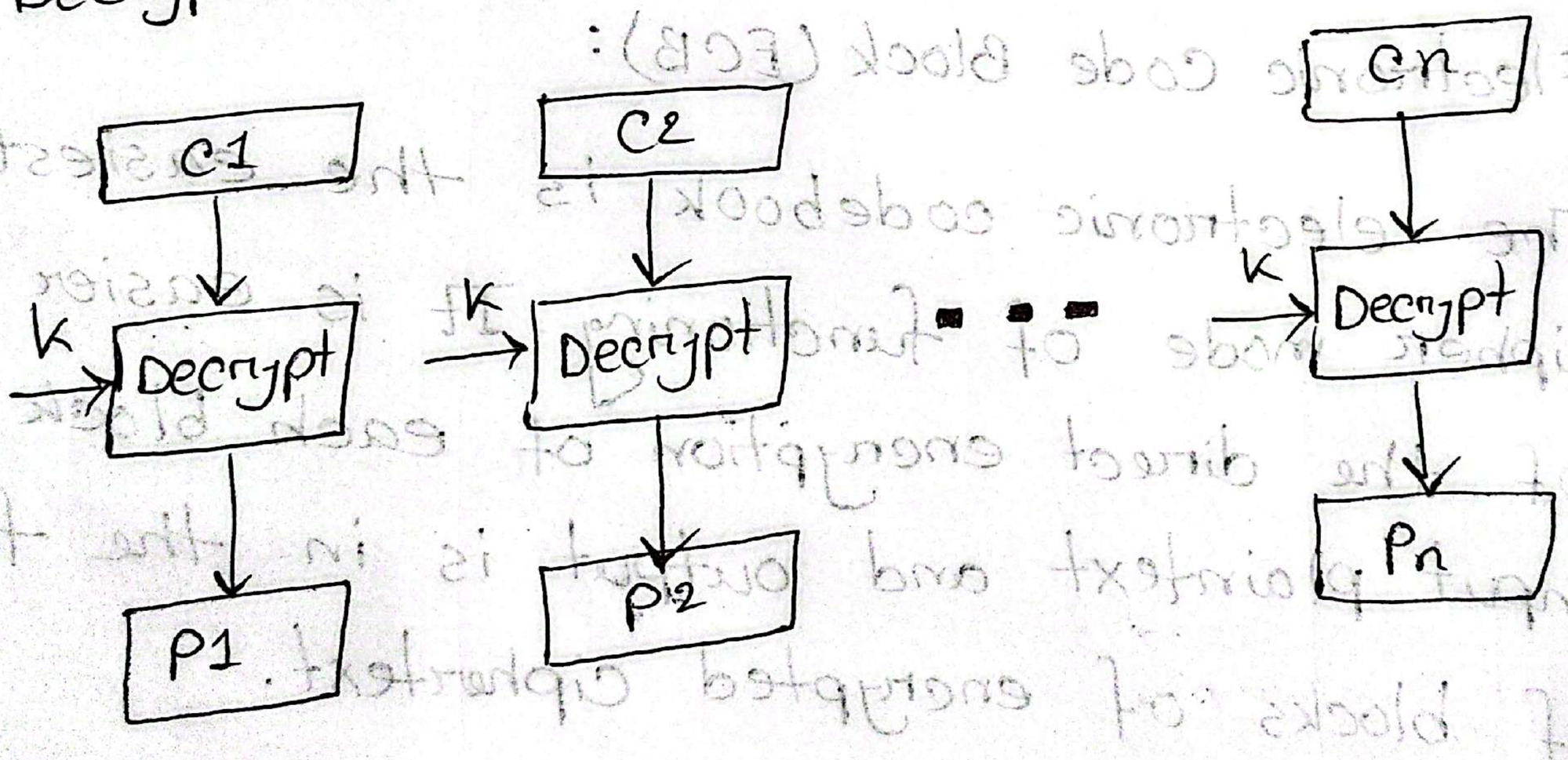
IT21059

Diagram:

Encryption →



Decryption →



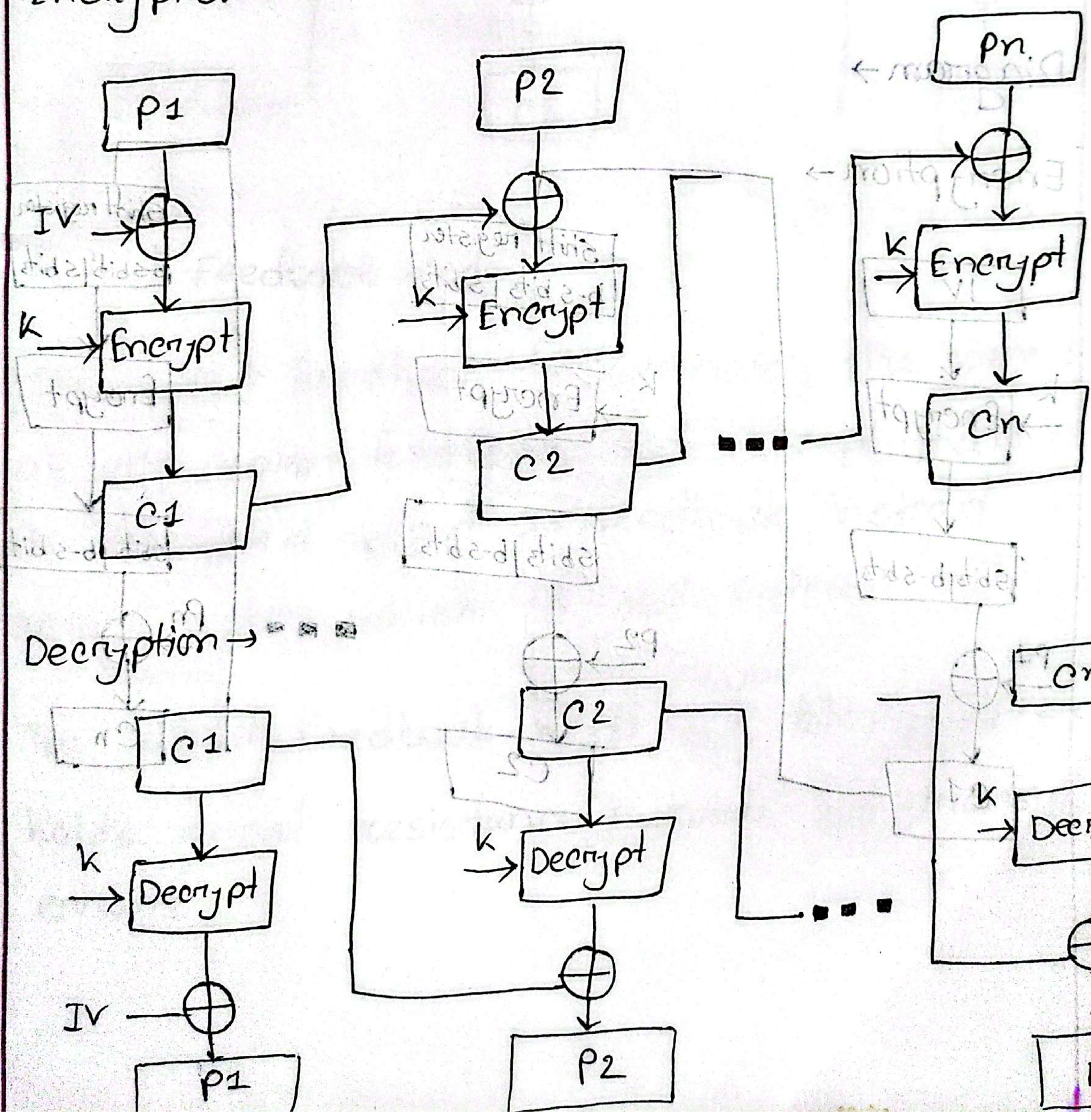
IT21059

Cipher Block Chaining (CBC)

Cipher Block chaining or CBC is an advancement made on ECB since ECB compromises some security requirements.

Diagram :

Encryption →

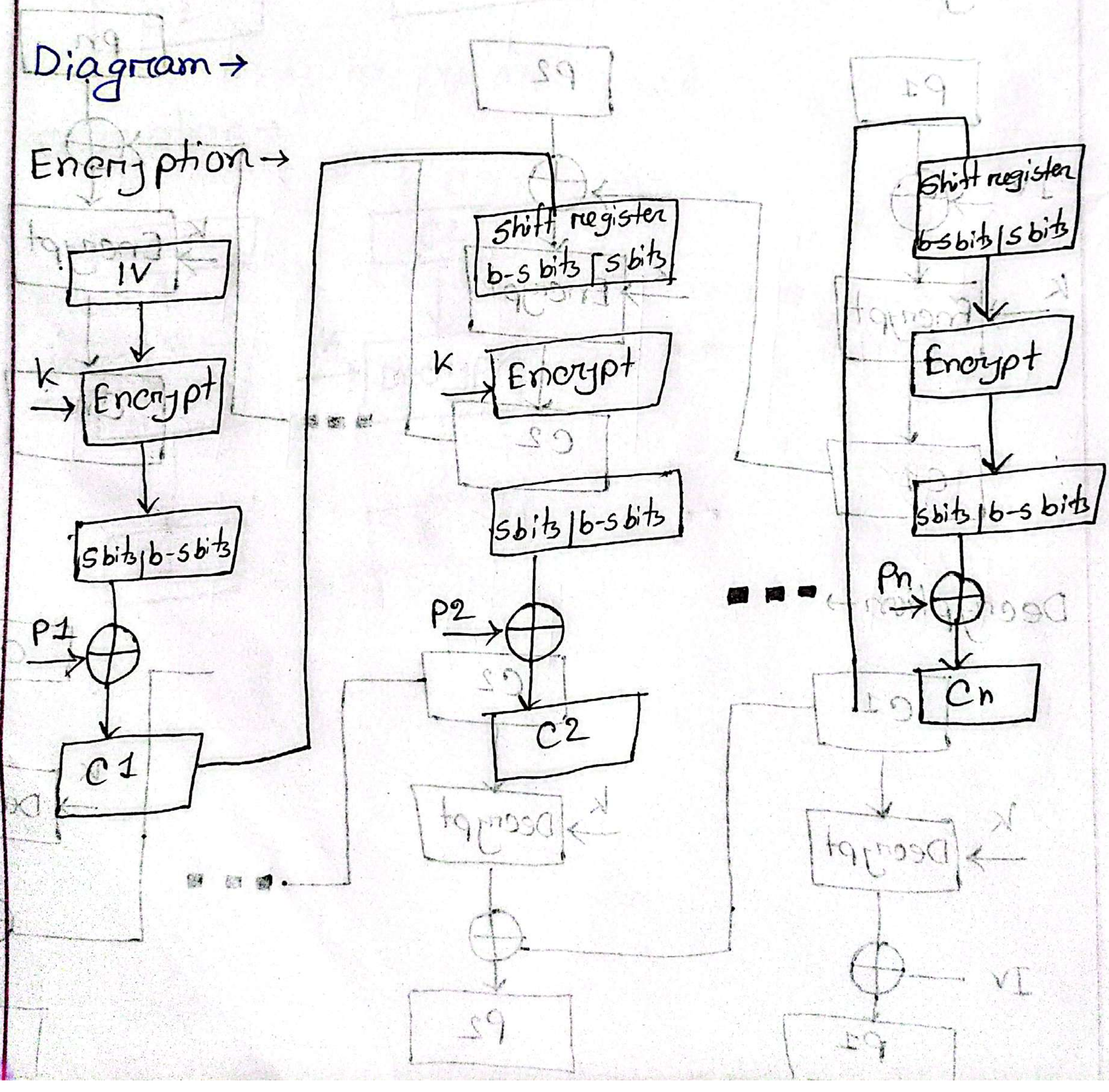


IT21059

Cipher Feedback Mode (CFM):

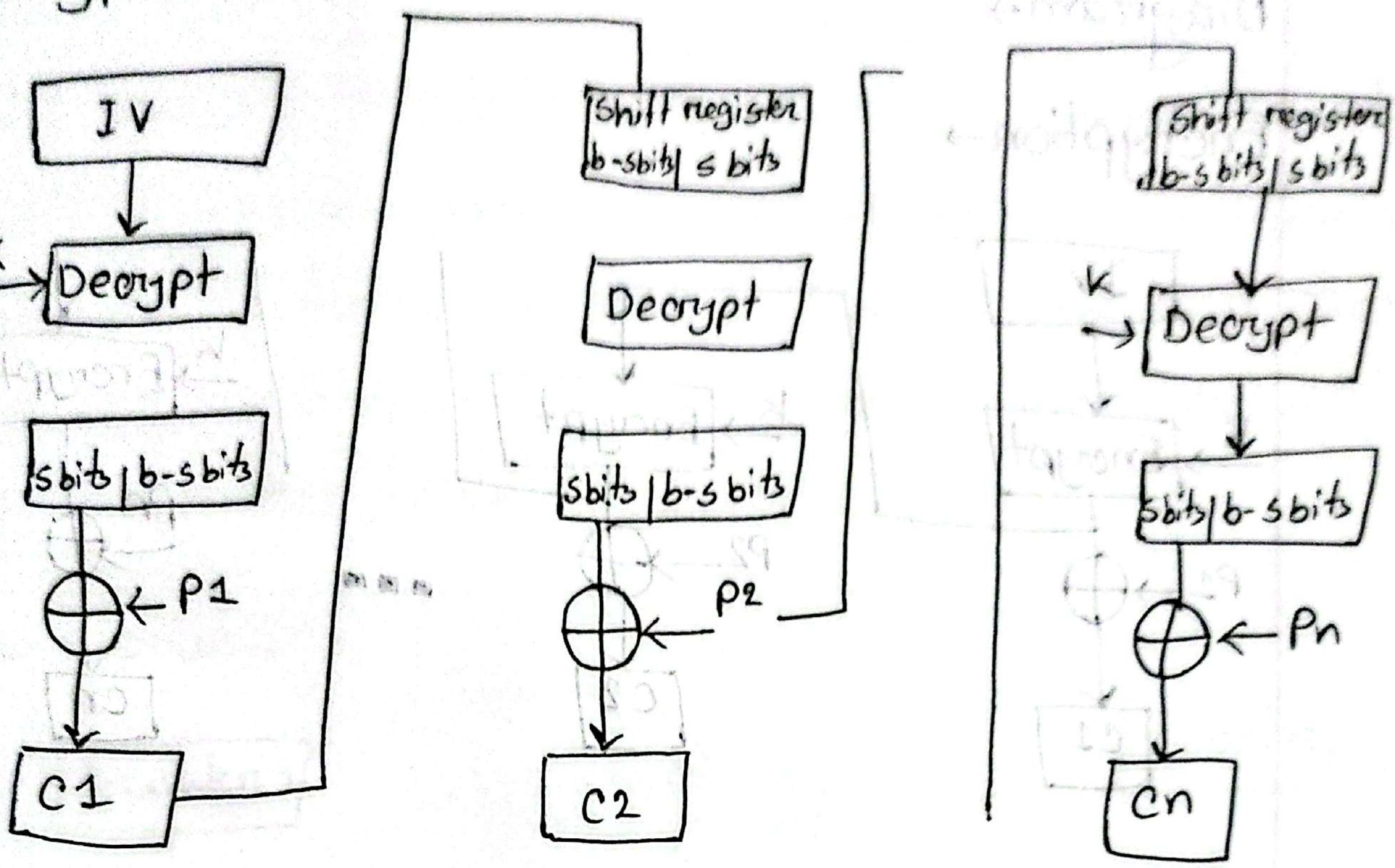
In this mode the cipher is given as feedback to the next block of encryption with some new specifications: first, an initial vector IV is used for first encryption and output bits are divided as a set of s and b-s bits.

Diagram →



IT21059

Decryption →



Output Feedback Mode

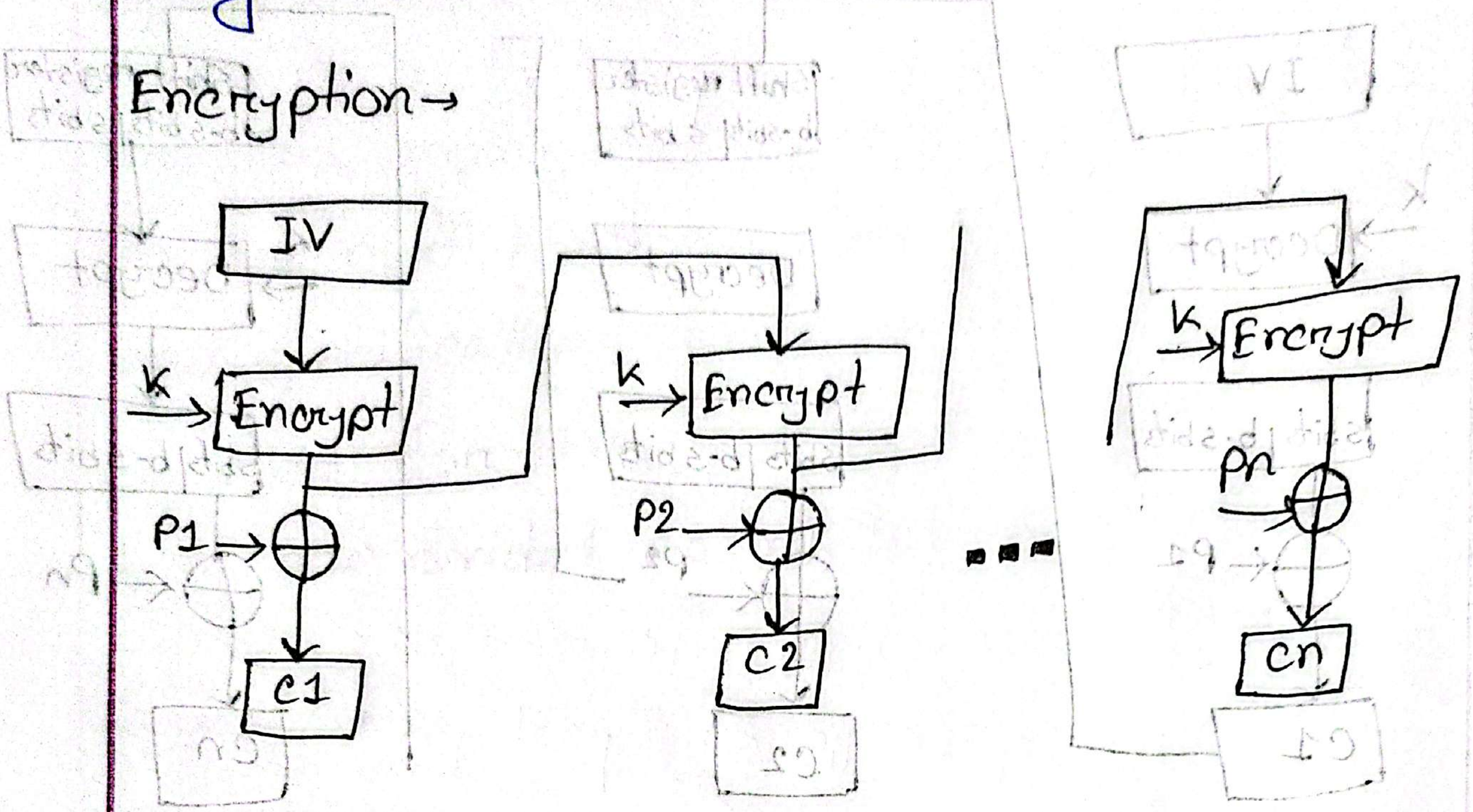
The output feedback follows nearly the same process as the cipher feedback mode except that it sends the encrypted output as feedback instead of the actual cipher which is XOR output.

The output feedback mode of block cipher holds great resistance towards bit transmission errors.

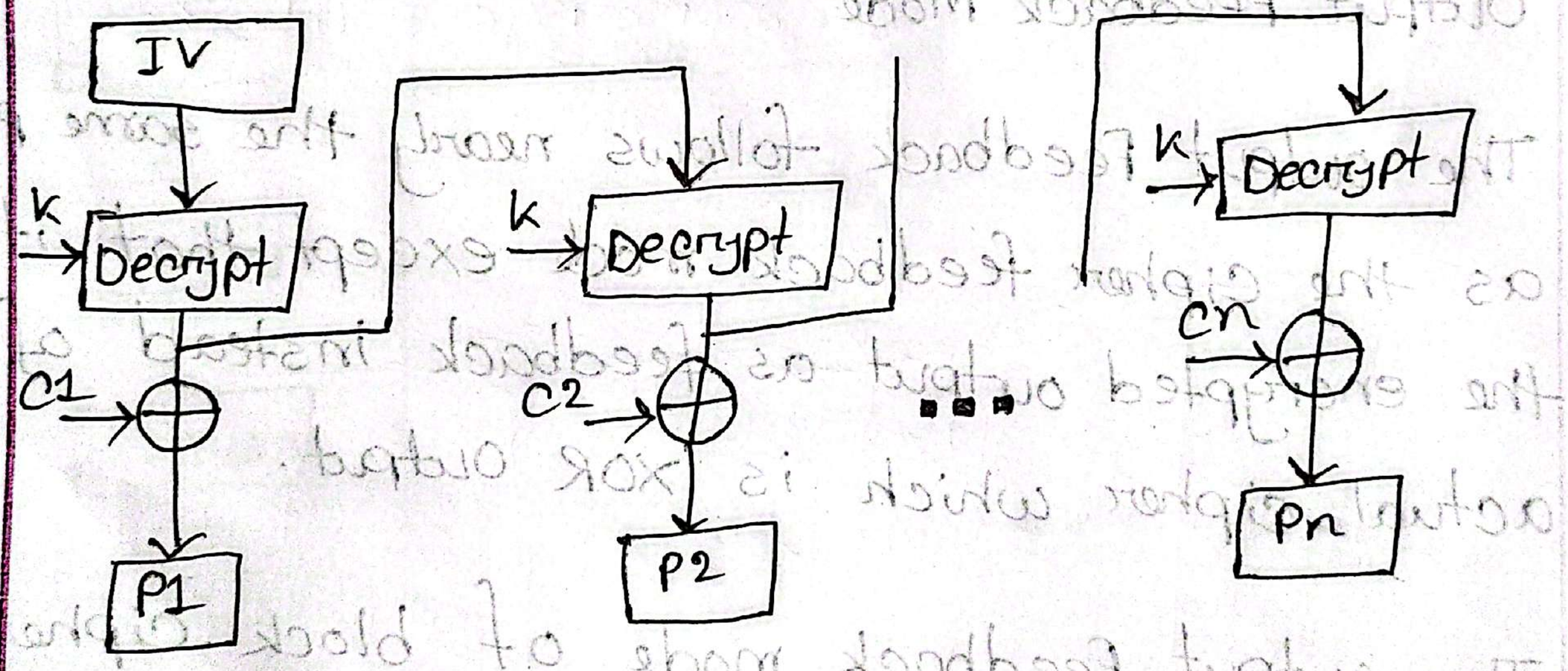
IT21059

Diagram →

Encryption →



Decryption →

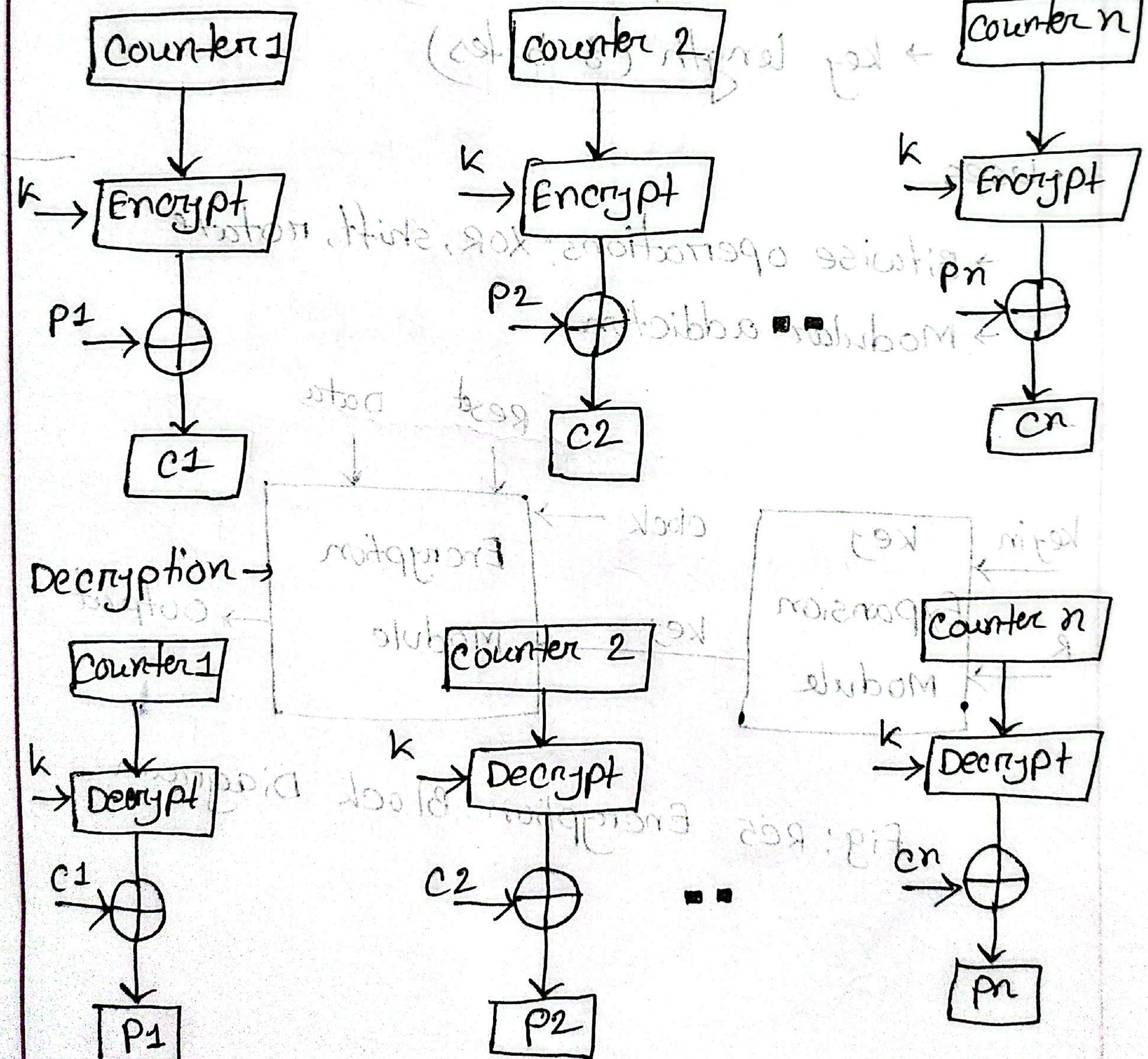


IT21059

Counter Mode →

The counter mode or CTR is a simple counter based block cipher implementation. Every time a counter initiated value is encrypted and given as input to XOR with plaintext which results in ciphertext block.

Encryption →



Decryption →

IT21059

Introduction of RC5:

RC5 is a fast, simple and secure symmetric key block cipher designed by Ron Rivest in 1994.

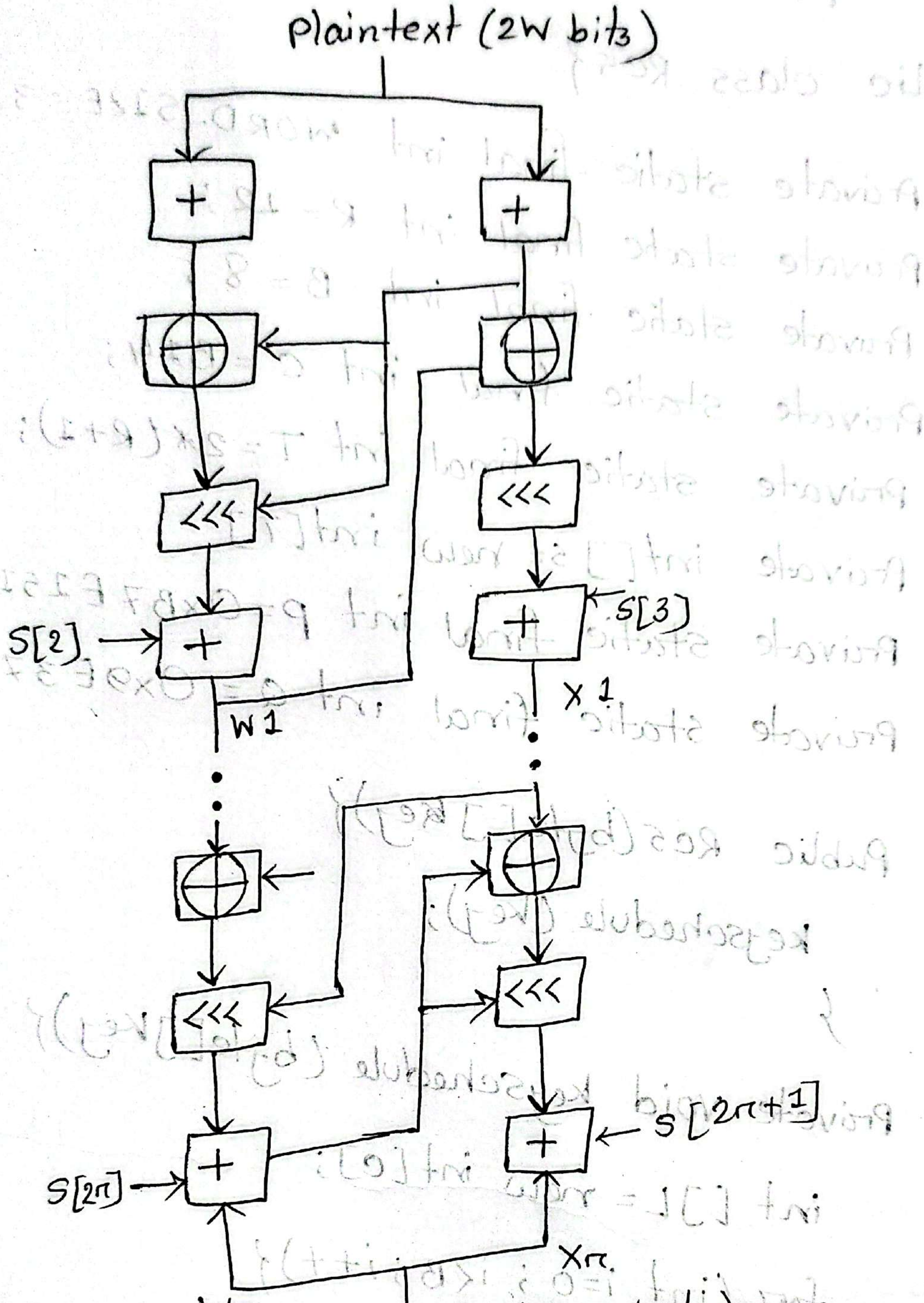
key features:

- Parameterizable
  - → word size (32 bits)
  - → Number of round (12)
  - → key length (8 bytes)
- Uses
  - → Bitwise operations: XOR, shift, rotate
  - → Modular addiction

Fig: RC5 Encryption Block Diagram

IT21059

RC5 Block Diagram :



IT21059

Java implementation

```
Public class RC5{

    Private static final int WORD-SIZE = 32;
    Private static final int R = 12;
    Private static final int B = 8;
    Private static final int C = B14;
    Private static final int T = 2*(R+1);
    Private int[] s= new int[T]
    Private static final int P=0xB7E15163;
    Private static final int Q = 0x9E3779139;

    Public RC5(byte[] key){
        keyschedule (key);
    }

    Private void keyschedule (byte[]key){
        int [] L= new int[c];
        for (int i=0 ; i<B ; i++){
            L[i/4] =(L[i/4] <<B) +(key[i] &0xff);
        }
```

```
S[0] = P;
for (int i = 1; i<T, i++){
 S[i] = S[i-1] + Q;
 }
int A=0, B=0, i=0, j=0;
for (int k=0; k<3*T; k++){
A = S[i] = Integer.rotateleft((S[i]+A+B),3);
B = L[j] = Integer.rotateleft((L[j]+A+B),(A+B));
i = (i+1) % T;
j = (j+1)%C;
}
}
Public int[] encrypt (int[] pt){
int A = pt[0] + s[0];
int B = pt[1] + S[1];
for (int i=1; i<=R; i++){
A = Integer.rotateLeft (A^B, B) + S[2*i];
B = Integer.rotateLeft (B^A, A) + S[2*i];
}
```

```
    return new int[]{A,B};
}
Public int[] decrypt (int[] ct){
    int B= ct[1];
    int A =ct[0];
    for (int i=k;i>=1;i--){
      B= Integer.rotateRight (B-S[2*i+1],A)^A;
      A= Integer.rotateRight (A-S[2*i],B)^B;
    }
A-=S[0];
B-= S[1];
return new(int[]){A,B};
}
Public static void main(string[]args){
    byte[]key= "password".getBytes();
    RC5 rc5 = new RC5(key);
    int[]Pt ={0X12345978, 0x9abcdef0};
    int[]ct = rc5.encrypt(Pt);
    System.out.Println("Encrypted: %.08x
      %.08x\n",ct[0],ct[1]);
```

IT21059

```
int[]dt = rcs.decrypt(ct);
System.out.print("Decrypted : %08x %08x\n",
                        dt[0],dt[1]);
}
}
```

Sample output:

Encrypted : 7f93d8c2 14236a29

Decrypted : 12345678 9abcdef0

Encryption →

- The input plainText is: 0x12345678 0x9abcdef0
- The encrypted ciphertext looks random
- After decryption, we get back the exact original plaintext.